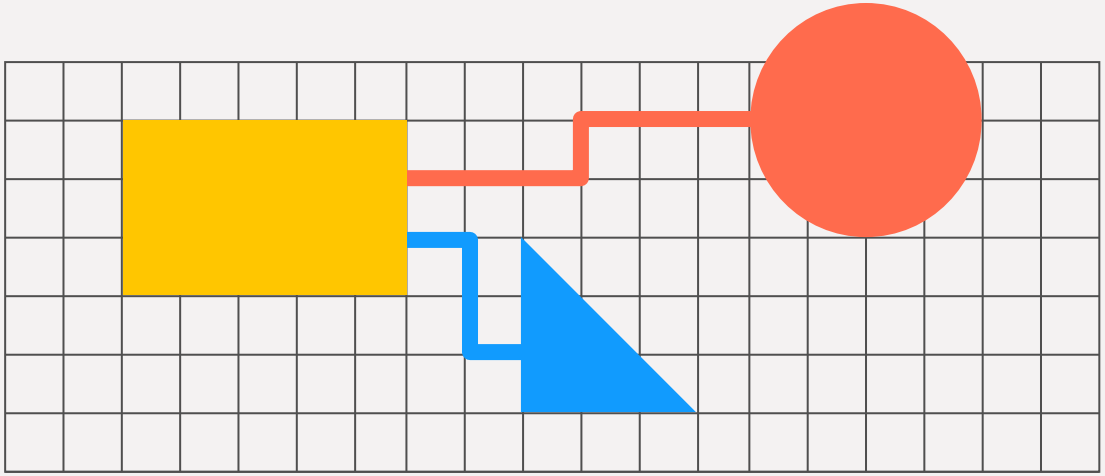
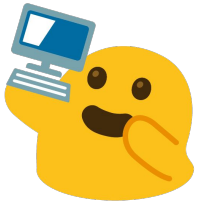
towards autonomy

agents and the new software paradigm



Google Cloud

hi, i'm mehul!



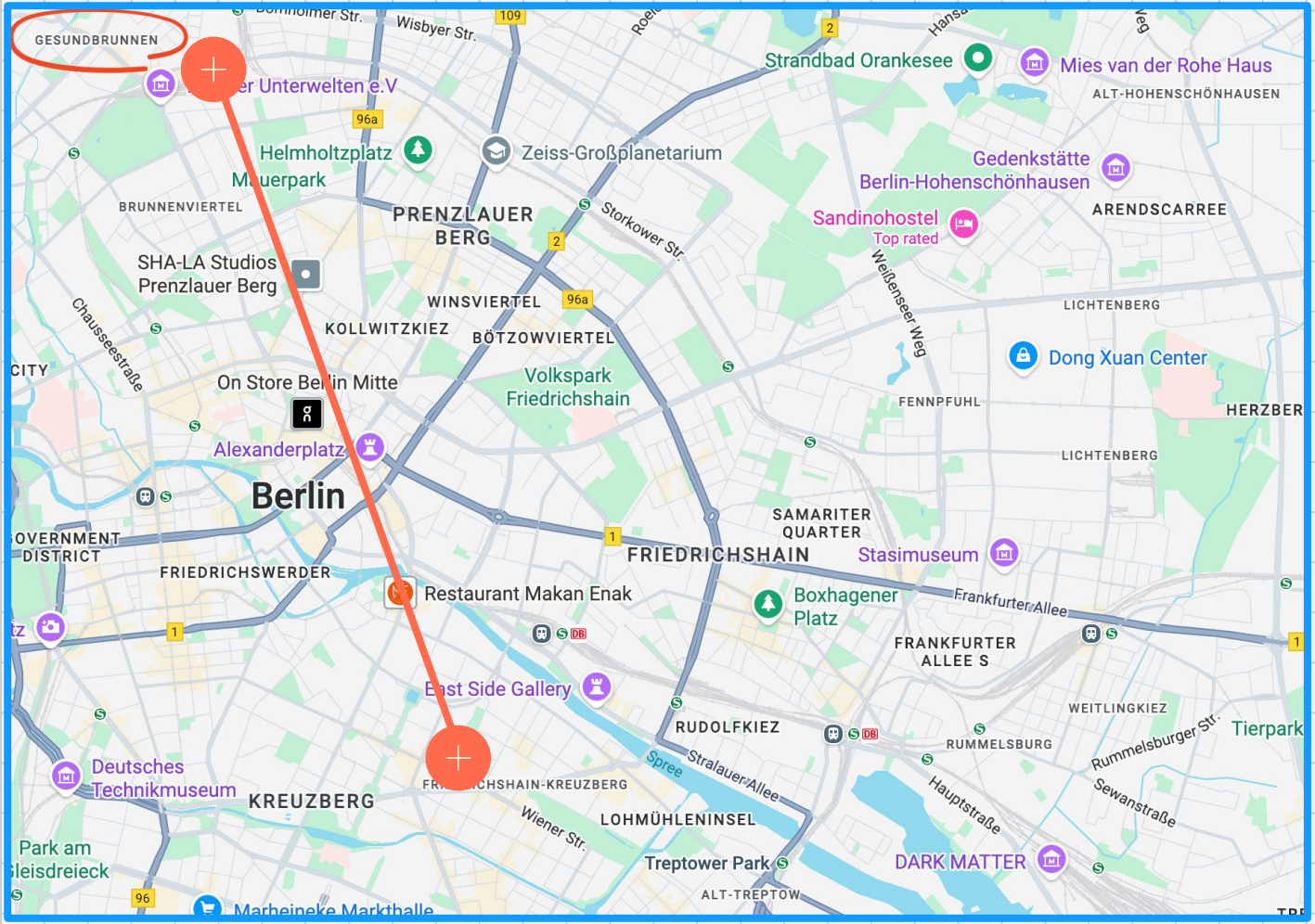
Sr. DevOps Engineer,
@BuildingMinds
📍 Berlin, Germany

Current focus: learning and building everything in AI....

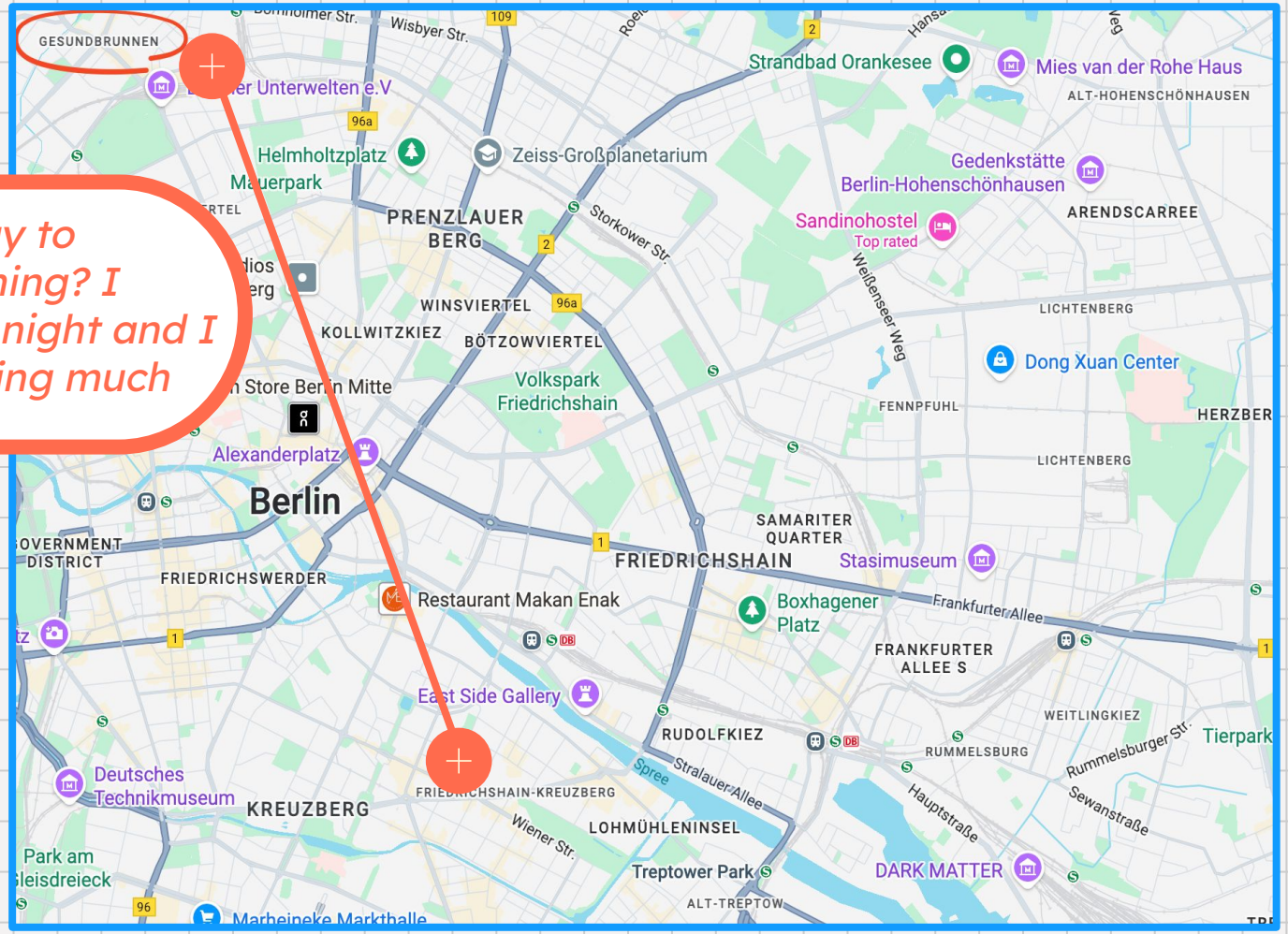
Community Hats: GDE for Google Cloud, Docker Captain

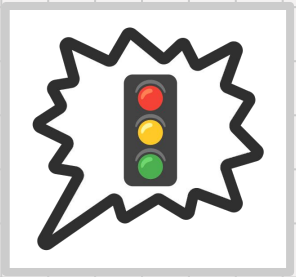
github.com/nomadicmehul

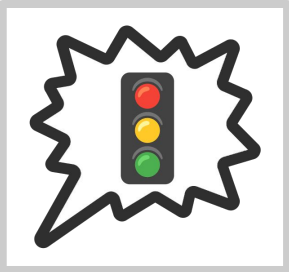
linkedin.com/in/nomadicmehul


GESUNDBRUNNEN
Wisbyer Str.
109
Strandbad Orankesee
Mies van der Rohe Haus
ALT-HOHENSCHÖNHAUSEN
96a
Helmholtzplatz
Zeiss-Großplanetarium
Gedenkstätte
Berlin-Hohenschönhausen
BRUNNENVIERTEL
PRENZLAUER
BERG
Storkower Str.
Sandinohostel
Top rated
ARENDSCARREE
SHA-LA Studios
Prenzlauer Berg
WINSVIERTEL
96a
LICHTENBERG
Chausseestraße
KOLLWITZKIEZ
BÖTZOWVIERTEL
Weißenseer Weg
Dong Xuan Center
Volkspark
Friedrichshain
On Store Berlin Mitte
FENNPFUHL
HERZBER
Alexanderplatz
LICHTENBERG
Berlin
SAMARITER
QUARTER
FRIEDRICHSHAIN
Stasimuseum
FRIEDRICHSWERDER
Restaurant Makan Enak
Boxhagener
Platz
Frankfurter Allee
FRANKFURTER
ALLEE S
East Side Gallery
WEITLINGKIEZ
RUDOLFKIEZ
RUMMELSBURG
Tierpark
Spree
Stralauer Allee
Deutsches
Technikmuseum
KREUZBERG
Hauptstraße
Sewanstraße
Wiener Str.
LOHMÜHLENINSEL
Park am
Gleisdreieck
Treptower Park
DARK MATTER
96
ALT-TREPTOW

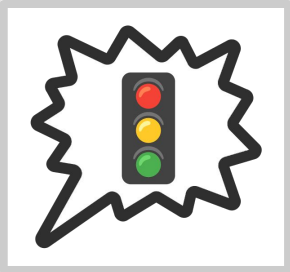
What's the best way to commute this morning? I stayed up late last night and I don't feel like walking much



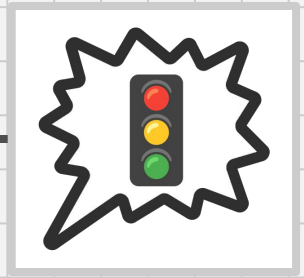


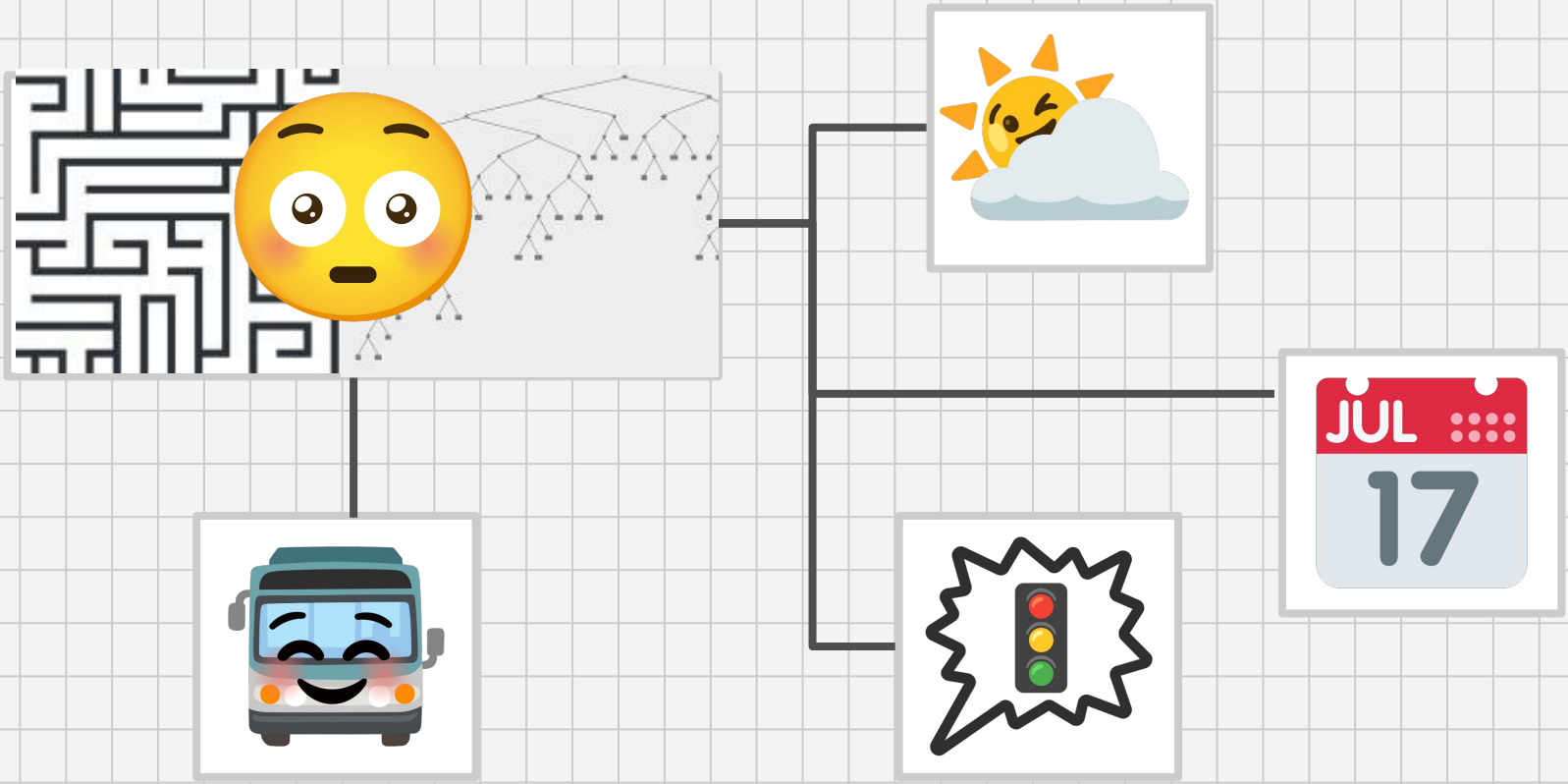


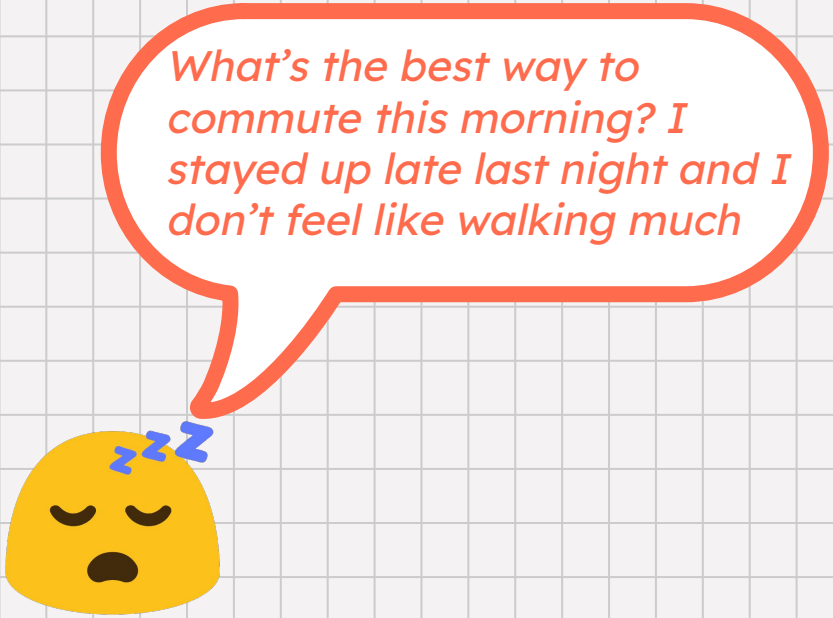

JUL
17



CommutePlanner
Traditional
Application

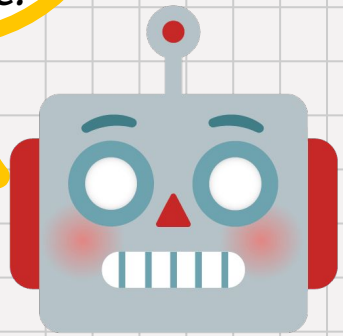



JUL
17

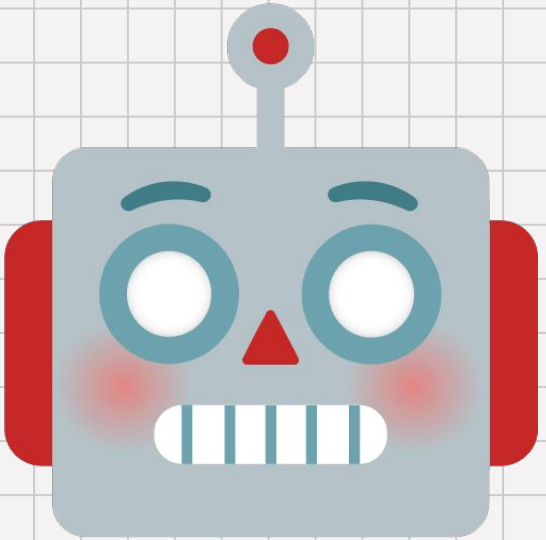

What's the best way to commute this morning? I stayed up late last night and I don't feel like walking much

What's the best way to commute this morning? I stayed up late last night and I don't feel like walking much

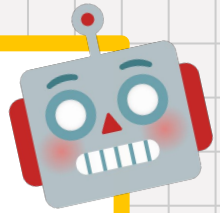
*Based on today's weather (chilly rain), your first meeting (not for another 2 hours), your pain flare-up, and the current **U-bahn** train status (normal), I suggest taking the **S-bahn**. Use the elevator at the Gleimstraße entrance.*



A generative AI **agent** is an application that uses **AI models** to make **decisions** and **act independently** in order to achieve a user-specified **goal**.

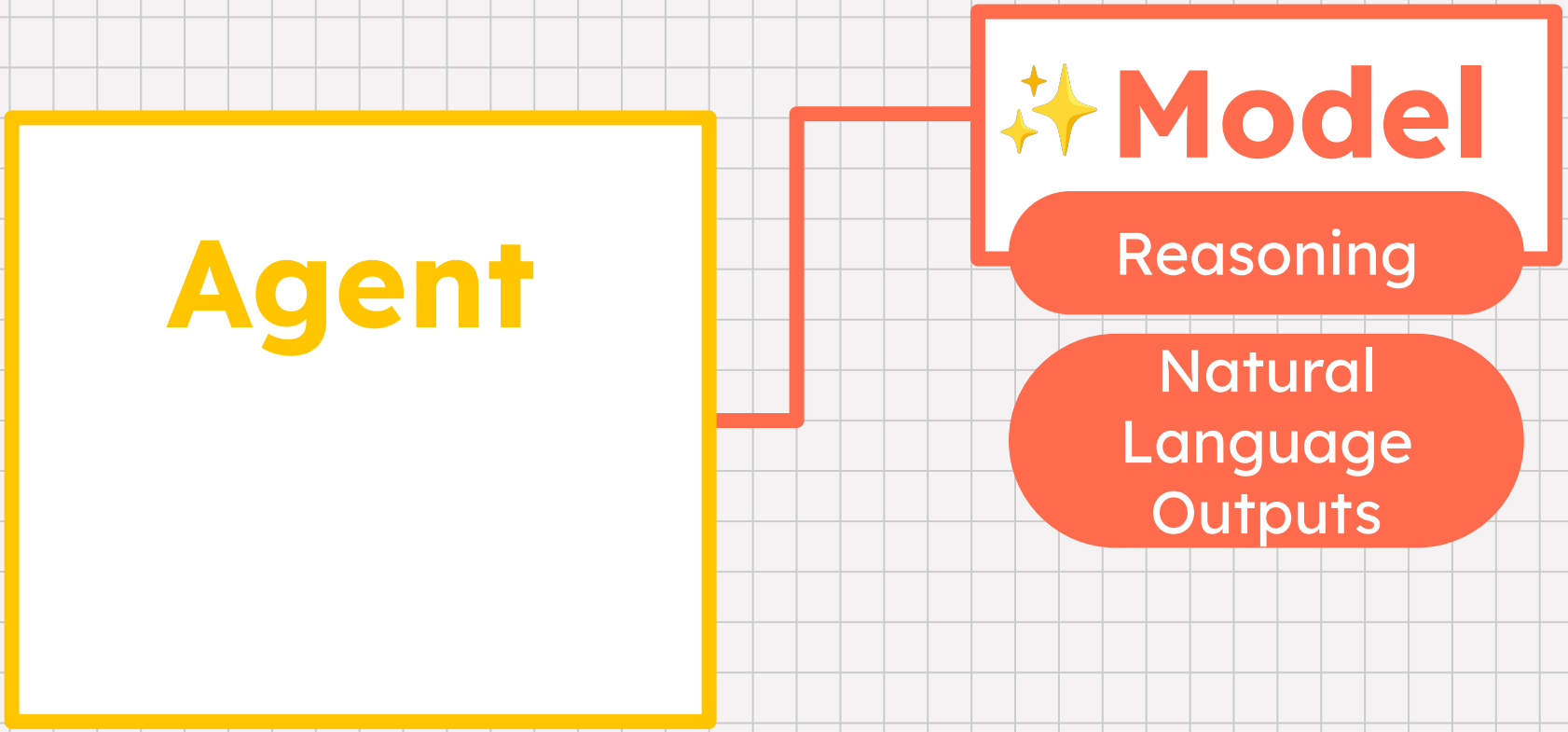


Agent



System Prompt

State, Memory


Agent
Model
Reasoning
Natural Language Outputs

Agent

Model

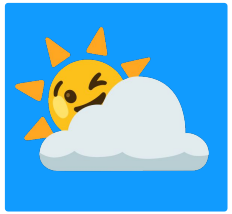
Tools

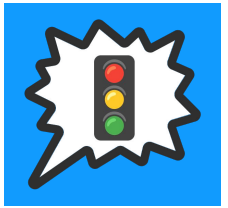
Agent

Agent Runtime

Model

Tools





What's the best way to commute this morning? I stayed up late last night and I don't feel like walking much

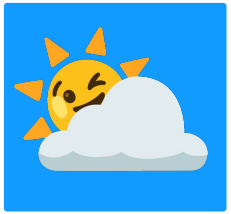
Agent

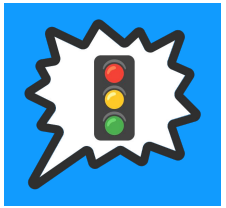
"You are a commute planner..."

session ID 78234331

Model

Tools





Agent

"You are a commute planner..."

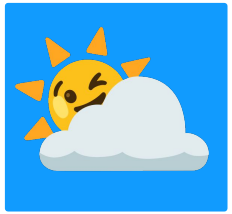
session ID 78234331

S-bahn might be a good option, check MTA alerts?

Model

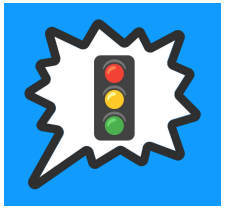
Tools

weather isn't great.



2 hours before first meeting!

S-bahn running normally. ✅



Agent

"You are a commute planner..."

session ID 78234331

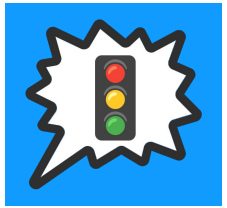
*Based on today's weather (chilly rain), your first meeting (not for another 2 hours), your pain flare-up, and the current **U-bahn** train status (normal), I suggest taking the **S-bahn**. Use the elevator at the Gleimstraße entrance.*

Model

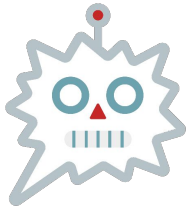
ok, feeling good about the response, here it is:

Tools



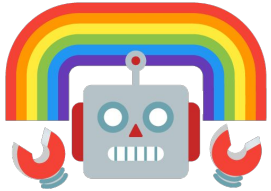


why agents?



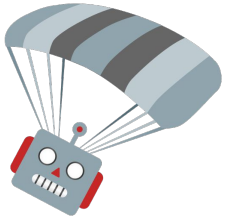
Flexible

Can **reason** based on personalized inputs, handle unexpected **edge cases**, and respond in **natural language. Outcome-driven** rather than path-driven.



Easy to Prototype

Reduced dev time **integrating** with external APIs and databases, and in building deterministic, “if this then that” systems.



Proactive

Can run an agent in response to a **user prompt,** on a **trigger** or **schedule** (autonomously).

when are agents a good fit?

I want a **personalized** study tool for college that can **learn** from my **preferences** and **performance**.

I want to set a **flexible** factory maintenance schedule based on **multiple real-time data sources.**

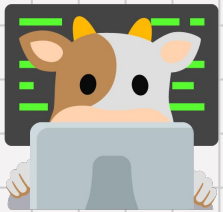
I want to **speed up** our customer refunds with an **autonomous** system.

(you don’t always need agents!)

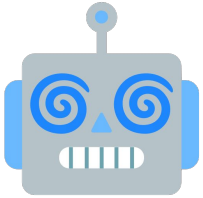
I’m doing **Q&A** against my enterprise data. I think a **RAG** architecture is enough!

I’ve got a **large prompt** that’s working well to **generate** marketing copy. I think I can get by with just **LLM calls.**

We run a **critical** financial transaction system that’s heavily **regulated**. We’re going to stick with our **ML classifier** (non gen AI), for now.

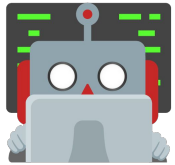


agent challenges



Predictability

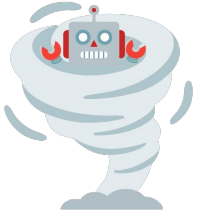
Agents use LLMs to reason and respond. **LLMs are nondeterministic.**



Stability

Complexity of using many tools, **error-handling, security, privacy**...

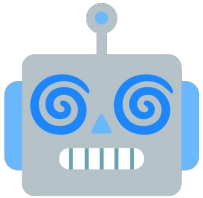
+ Framework + protocol landscape is **evolving rapidly.**



Operations

Tracking calls through a **distributed system** is hard, debugging is tricky, tracking **token** throughput is important (**costs**), **quota,** LLM reasoning can be **opaque**...

how to mitigate?



Predictability

✅ choose reasoning **models**, when possible. (eg. gemini 2.5 flash)

✅ **ground** LLM in trusted data to reduce hallucinations (RAG, search)

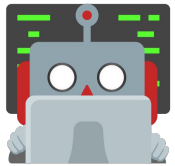
✅ guide your agent to **think step by step** (chain of thought or few-shot prompting)

✅ use **state management** and **memory.**

✅ implement **checks** in the agent's workflow: tool-call request validation, safety filters, logic checks...

✅ **test** and **evaluate** your agent.

how to mitigate?



Stability

Model Context Protocol

✅ use consistent tool abstractions via **Model Context Protocol (MCP)**

✅ when writing your agent's **system prompt,** be as **detailed** as possible. (tool descriptions)

✅ implement or leverage **error-handling** in your agent (circuit-breakers, retries, hand off to a human in the loop...)

✅ utilize **state management,** especially the workflow status field (recover from interruptions, prevent infinite loops)

✅ be willing to **refactor** and **adjust** with new framework features, tool interfaces...

how to mitigate?

✅ in the dev phase, **log** verbosely.

✅ build robust **test** and **evaluation** datasets. automate these tests on every commit.

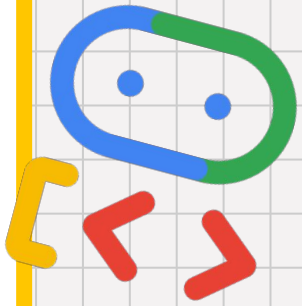
✅ gather necessary **metrics**, like token throughput to your agent's models, and response code distribution. create SLOs and **alerts** if these trip certain thresholds.

✅ leverage agent framework's **tracing** features to track model and tool calls, find + address **latency bottlenecks.**

Operations

Agent Development Kit

Discover, build and deploy agents



Agent Development Kit

Client side SDK to define multi-agent applications for complex, real world scenarios

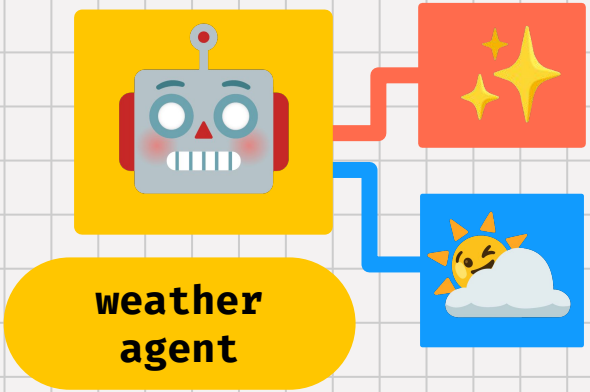
Agents	Tool	Orchestration	Callbacks
Bidirectional Streaming	Session Management	Evaluation	Deployment
Artifact Management	Memory	Code Execution	Planning

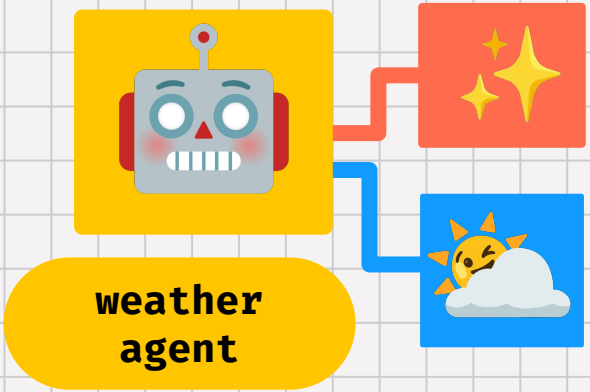
Debugging

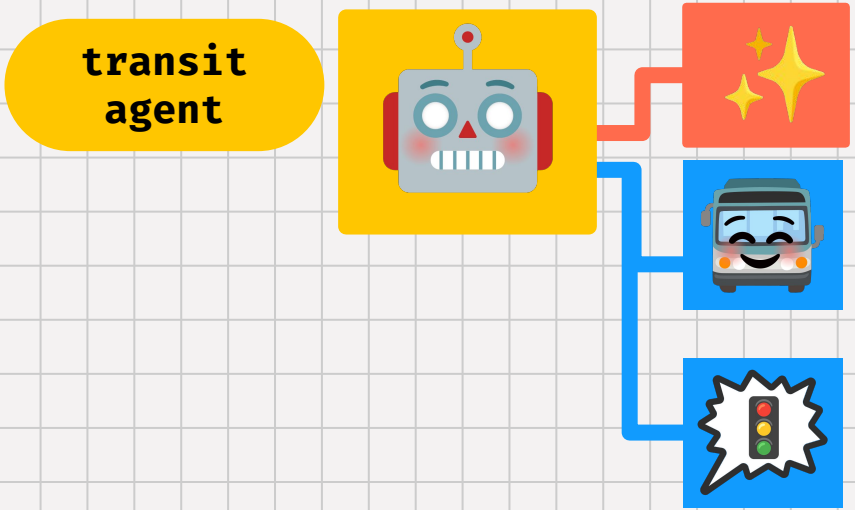
Trace

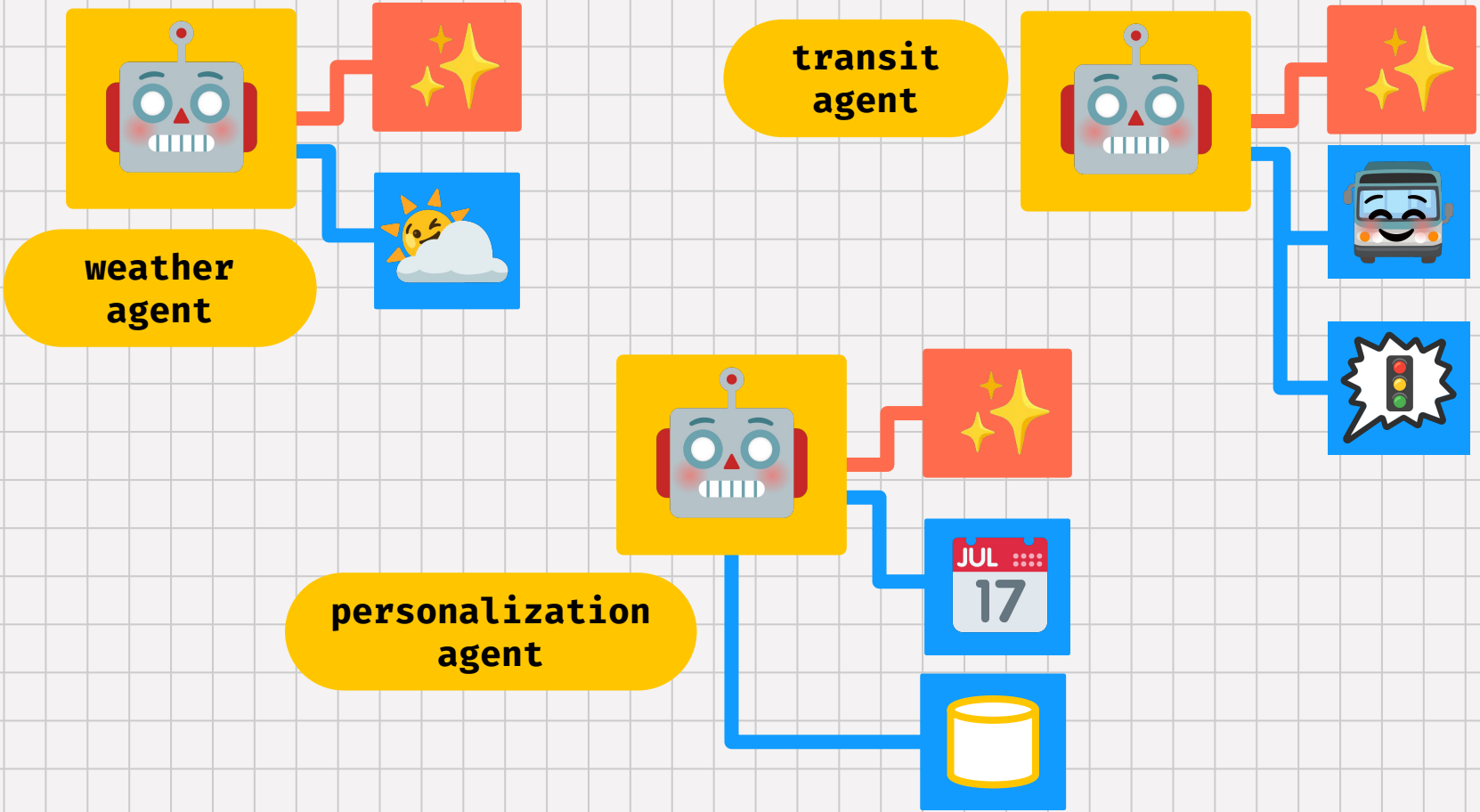
Models

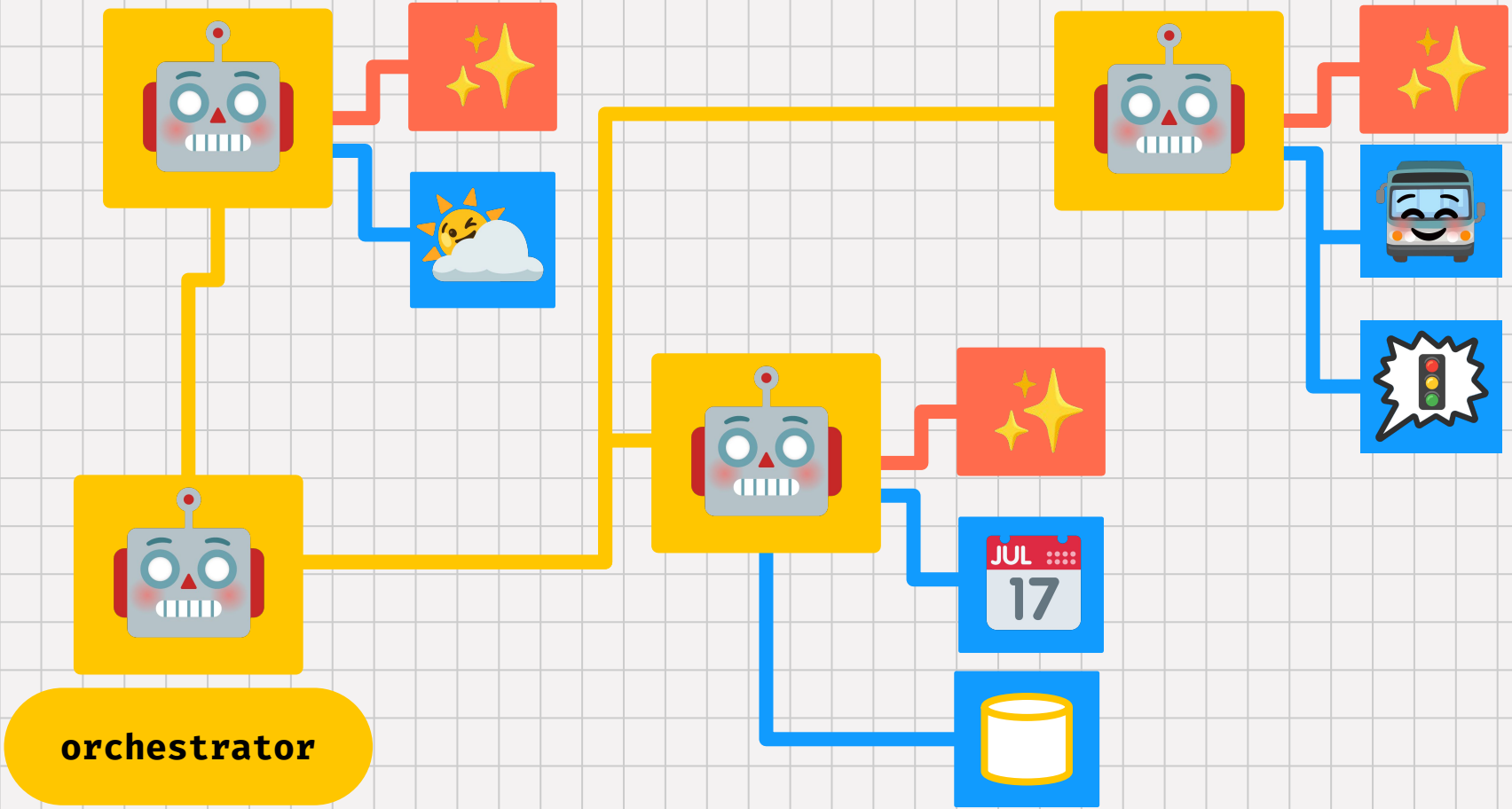
Agent


weather
agent


weather
agent


transit
agent

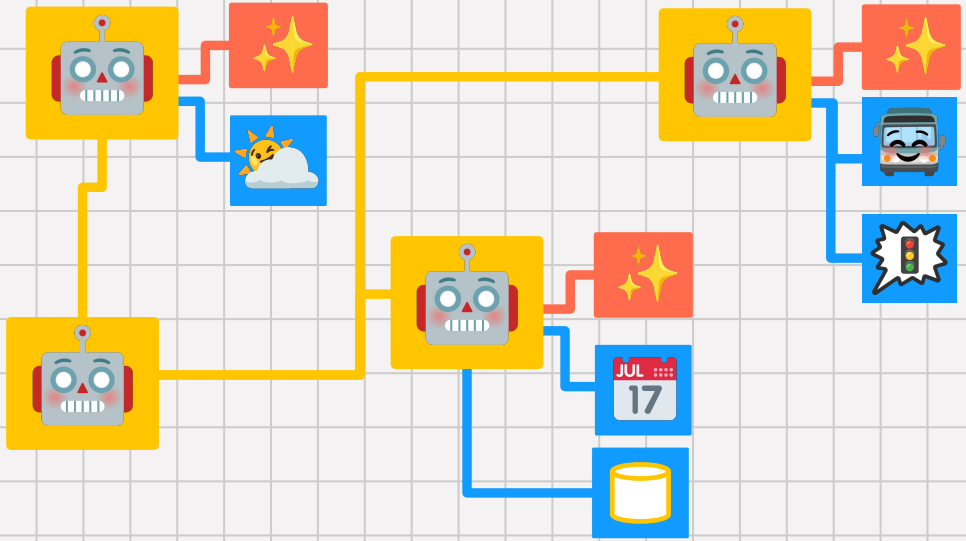

weather agent
transit agent
personalization agent
JUL
17


JUL
17
orchestrator

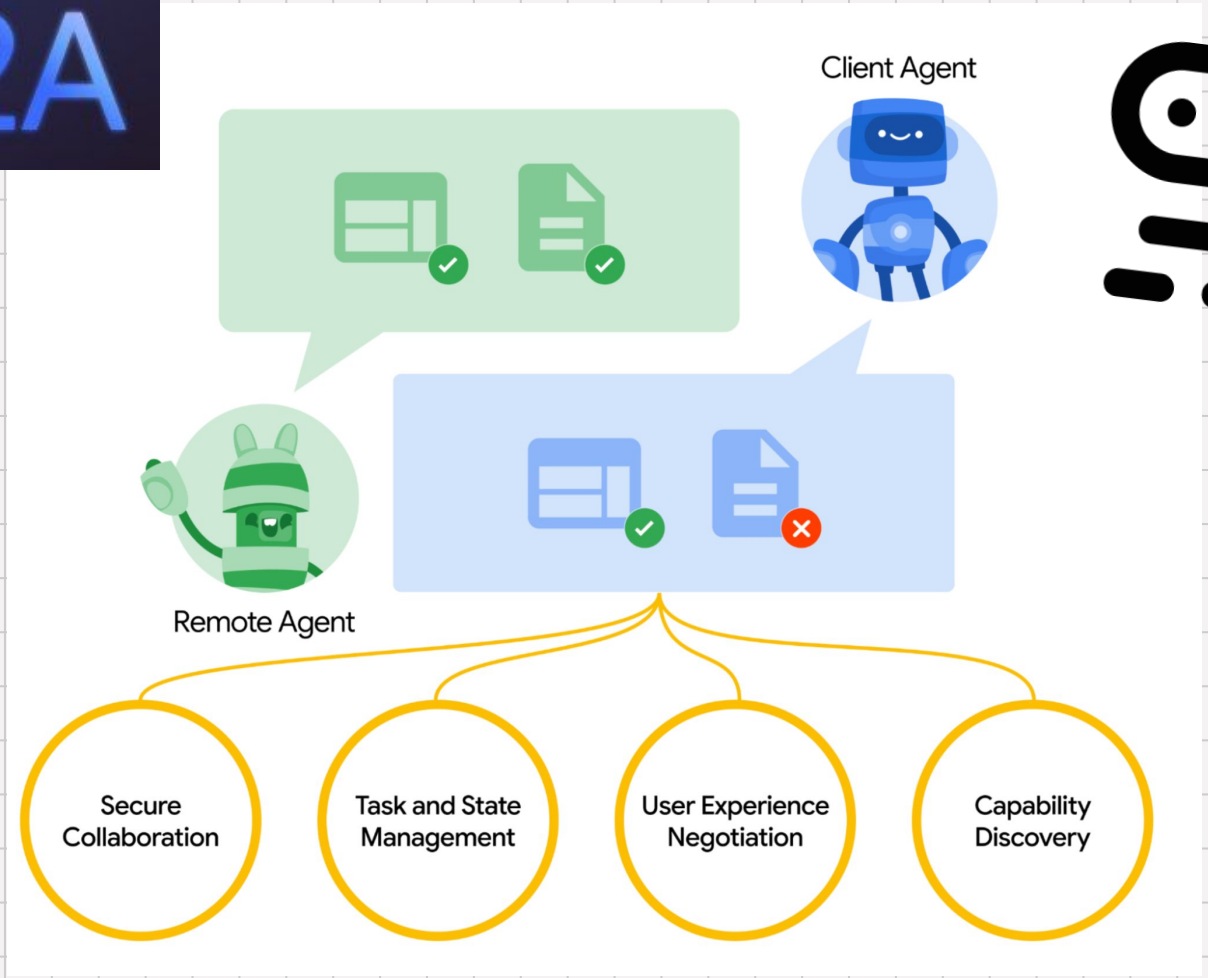
multi-agent systems: benefits

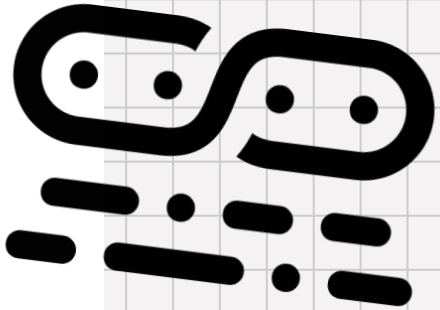
#1: system of **experts**, each with their own **prompts** and reasoning **flows** ➡️ **higher-quality** responses.

#2: modular architecture ➡️ easier to **scale** separately,
test separately,
share and **re-use** the code.

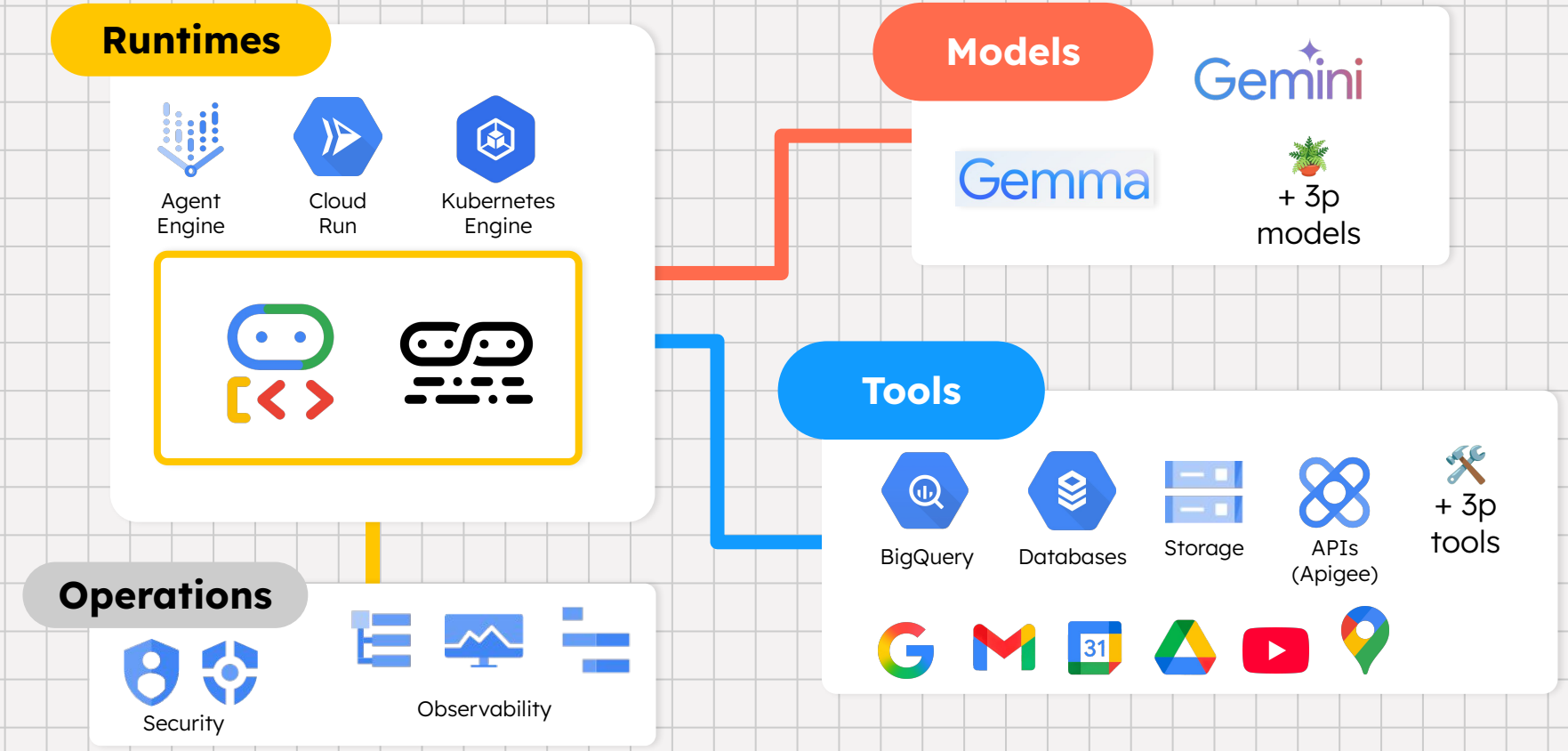


A2A

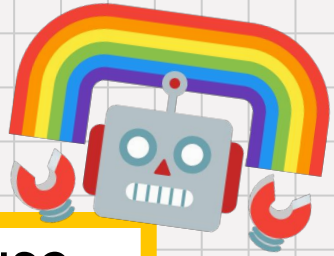

Client Agent
Remote Agent
Secure Collaboration
Task and State Management
User Experience Negotiation
Capability Discovery



build agents with Google Cloud



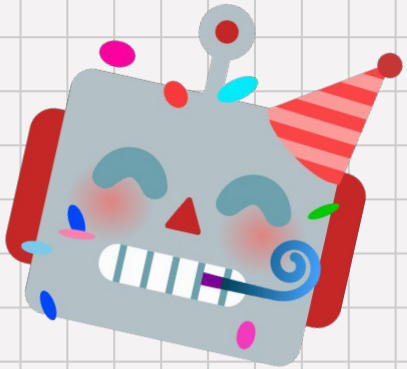
recap!



💡 AI agents unlock **flexible, personalized** software use cases by taking a **goal-oriented** approach, using **AI models** and external **tools.**

💡 AI agents represent a move towards **autonomous** software systems, beyond determinism and LLM prompting.

💡 Google provides multiple open tools for agent development (**ADK**, **A2A**), alongside the power of **Google Cloud.**



go forth and build agents!

credits to [Emoji Kitchen](https://emojikitchen.dev/) for the images!